

Exerciții — Strategy & Observer

Caiet de practică · C# · hint-free

Cum lucrezi cu caietul ăsta

Sunt 6 exerciții: 3 pe **Strategy**, 3 pe **Observer**, gradate ([**usor**], [**mediu**], [**provocare**]). Nu-ți mai dau pași — ai făcut deja câte două ghidate în repo (**ex1–ex4**). Aici primești doar **problema**, **contractul** și **ce trebuie să iasă**; tu proiectezi clasele, contextul și testul.

Pentru fiecare, lucrează într-un folder nou (**ex5**, **ex6**, ...) sau într-un proiect separat — cum îți e comod. Reguli valabile peste tot:

- Fără **is**, **as**, **GetType()**. Fără **switch/if** pe tip în context/subiect.
- Fără **List/Dictionary/LINQ** (excepție: **string.Join** unde ajută) — array-uri brute și **for**.
- Mesajele de eroare în engleză.
- Fiecare exercițiu are un **Testare** chemat din **Program.cs**, ca la celelalte.

Definiția de „gata” e la finalul fiecărui exercițiu: dacă output-ul arată acolo, ai terminat.

Teoria e în **TEORIE.pdf** — Strategy la lecția 1, Observer la lecția 2. Recitește-o dacă te blochezi, dar încearcă întâi singur.

Partea I — Strategy

Reține forma: o familie de algoritmi interschimbabili în spatele unui contract; contextul ține unul și îl întreabă, fără să știe care e.

S1 — Validator de parolă [usor]

Context. Un câmp de parolă verifică dacă o parolă respectă o **politică**. Politica se poate schimba (site-uri diferite, cerințe diferite), fără să rescrii câmpul.

Contract.

```
public interface IPoliticaParola
{
    string Nume { get; }
    bool EsteValida(string parola);
}
```

Ce construiești.

Clasă	Regula
PoliticaSimpla	minim 6 caractere
PoliticaMedie	minim 8 caractere și conține cel puțin o cifră
PoliticaPuternica	minim 8 caractere, o cifră, o literă mare și un caracter care nu e literă/cifră
CampParola (Context)	ține o IPoliticaParola; Verifica(string parola) întoarce true/false; SchimbaPolitica(...)

Test. Aceeași parolă (ex: "abc123") verificată cu toate 3 politicile; apoi una puternică (ex: "Abc123!x"). Afișează numele politicii și rezultatul.

Gata când vezi aceeași parolă acceptată de politica simplă și respinsă de cea puternică, doar schimbând strategia — fără niciun if pe „ce fel de politică” în CampParola.

S2 — Comision de vânzare [mediu]

Context. La o vânzare se calculează comisionul agentului. Firma schimbă des modul de calcul; nu vrei să atingi clasa Vanzare de fiecare dată.

Contract.

```
public interface IComision
{
    string Nume { get; }
    decimal Calculeaza(decimal valoareVanzare);
}
```

Ce construiești.

Clasă	Formula
<code>ComisionFix</code>	o sumă fixă, dată în constructor (ex: 50), indiferent de valoare
<code>ComisionProcent</code>	un procent din valoare, dat în constructor (ex: 10%)
<code>ComisionPePraguri</code>	5% pentru partea până la 1000, 10% pentru cât depășește 1000
<code>Vanzare (Context)</code>	ține valoarea și o <code>IComision</code> ; <code>Comision()</code> întoarce suma; <code>SchimbaComision(...)</code>

Test. O vânzare de 1500, calculată cu toate 3 strategiile. Verifică manual `ComisionPePraguri`: $5\% \cdot 1000 + 10\% \cdot 500 = 100$.

Gata când cele 3 strategii dau 3 valori diferite pe aceeași vânzare, iar `ComisionPePraguri` dă 100 la 1500.

S3 — Reducere compusă [provocare]

Context. Extinzi S2. Vrei să aplici o reducere PESTE un comision deja calculat — și eventual încă una peste aceea.

Ce construiești. O strategie `ComisionCuPlafon` care primește în constructor **altă** `IComision` și o valoare maximă, și întoarce $\min(\text{comision_interior}, \text{plafon})$. Apoi o `ComisionCuBonus` care primește altă `IComision` și adaugă o sumă fixă.

Test. Compune-le: `ComisionCuPlafon(200, ComisionProcent(20%))` pe o vânzare de 1500 $\rightarrow \min(300, 200) = 200$. Apoi împachetează rezultatul într-un `ComisionCuBonus(50, ...)` $\rightarrow 250$.

Gata când poți înlănțui strategii care se împachetează una pe alta și rezultatul se calculează corect de la interior spre exterior.

Întrebare. O strategie care primește altă strategie de același tip și o „îmbunătățește”... îți sună cunoscut din `ex1` (bonusul `LivrareCuReducere`)? Asta e un **Decorator** deghizat — îl vom numi așa la lecția 4.

Partea II — Observer

Reține forma: o sursă își schimbă starea și îi anunță pe toți abonații, fără să știe cine sunt sau ce fac. Nu așteaptă niciun răspuns înapoi.

O1 — Canal cu abonați [usor]

Context. Un canal publică videoclipuri. Când apare unul nou, toți abonații sunt anunțați — fiecare în felul lui.

Contract.

```
public interface IAbonat
{
    void Notifica(string titluVideo);
}
```

Ce construiești.

Clasă	La Notifica("X")
AbonatEmail	[EMAIL] Video nou: X
AbonatPush	[PUSH] X
Canal (Subject)	ține IAbonat [] prin constructor; PublicaVideo(string titlu) anunță toți abonații

Bonus. Aboneaza/Dezaboneaza care cresc/micșorează array-ul manual (fără List). Atenție la dimensionarea array-ului la Dezaboneaza — numără întâi câți rămân, nu presupune că scoți exact unul. (Da, e capcana din review-ul tău la ex2.)

Gata când o publicare declanșează reacția tuturor abonaților, iar după un Dezaboneaza cel scos nu mai primește nimic.

O2 — Cont bancar [mediu]

Context. La fiecare tranzacție pe cont, mai mulți observatori reacționează — dar unul reacționează doar uneori.

Contract.

```
public interface IObservatorCont
{
    void Actualizeaza(decimal soldNou);
}
```

Ce construiești.

Clasă	Reacția
NotificatorSms	[SMS] Sold nou: X la fiecare schimbare
JurnalAudit	[AUDIT] Sold inregistrat: X la fiecare schimbare

Clasă	Reacția
AlertaSoldMic	afișează [ALERTA] Sold sub prag: X numai dacă soldNou < prag (pragul dat în constructor); altfel tace
Cont (Subject)	ține soldul și IObservableCont[]; Depune(decimal) și Retrage(decimal) schimbă soldul și anunță observatorii

Test. Pornești de la 1000, faci Retrage(950) (sold 50 → alerta se declanșează), apoi Depune(500) (sold 550 → alerta tace). Retrage care ar duce soldul sub 0 aruncă InvalidOperationException.

Gata când SMS și audit reacționează la fiecare mișcare, dar alerta apare **doar** când soldul scade sub prag.

O3 — Licitatie cu auto-licitator [provocare]

Context. La o licitație, fiecare ofertă nouă îi anunță pe toți participanții cu noul preț. Un participant special re-licitează automat dacă e depășit.

Contract.

```
public interface IParticipant
{
    void OfertaNoua(decimal pretCurent);
}
```

Ce construiești.

Clasă	Reacția
Spectator	[SPECTATOR] Pret curent: X
AutoLicitator	are un pretMaxim (din constructor); dacă pretCurent < pretMaxim, plasează o ofertă nouă de pretCurent + 10 înapoi în licitație
Licitatie (Subject)	ține prețul curent și IParticipant[]; Liciteaza(decimal suma) acceptă doar sume mai mari decât prețul curent (altfel ArgumentException), setează prețul și anunță toți participanții

Atenție la bucla de feedback. AutoLicitator cheamă Liciteaza din interiorul lui OfertaNoua — deci o ofertă declanșează alte oferte. Gândește-te: ce oprește lanțul? (răspuns: pretMaxim — când nimeni nu mai poate depăși, notificările nu mai produc oferte noi). Ai grijă ca licitatorul să nu se supraliciteze pe sine.

Gata când o singură Liciteaza de la un spectator declanșează o „bătălie” automată care se oprește singură când se atinge pretMaxim, și vezi în output cum urcă prețul pas cu pas.

Întrebare. La O2, observatorii doar *reacționează*. La O3, un observator *declanșează o nouă schimbare în subiect*. Ce risc apare când observatorii pot modifica sursa pe care o ascultă, și de ce trebuie o condiție de oprire?